

SENTINEL: LSM-eBPF Co-Design with Graph-Structured LLM Inference for Zero-Trust APT Detection

Juharasha Shaik  *Independent Researcher*

Fremont, CA 94538, USA

Email: shaik.juharasha@gmail.com

ORCID: <https://orcid.org/0009-0003-7291-360X>

Abstract—Contemporary host-based intrusion detection systems (HIDS) suffer from a critical semantic gap: they observe *what* a process does but cannot reason about *why* it does so. Rule-based eBPF engines such as Falco attach to tracepoints that fire *before* the kernel resolves the final object, leaving them vulnerable to time-of-check-to-time-of-use (TOCTOU) races and path-aliasing attacks. We present SENTINEL, an IPG-centric forensic architecture that addresses these limitations by utilizing an Intent Provenance Graph (IPG) as a unified intermediate representation (IR) for host-level activity. A deterministic `provenance_score()` graph scorer acts as the primary detector; a local LLM classifies gray-zone windows ($0.15 < g < 0.55$) and produces MITRE ATT&CK attribution, while clear benign/malicious windows bypass LLM inference. CoVe-verified LLM outputs and an LTL symbolic guardian enforce hard safety floors and prevent evasions. Evaluated on DARPA TC E3 CADETS, hybrid graph-first fusion achieves behavioral-only $F1 = 0.603$ (LLM-only v4: 0.597) and TI-aided $F1 = 0.915$; on a 15-trace attack trace subset, IPG reduces tokens by 57.5% at window size 20 (83.3% full trace) using local Ollama Llama-3.1-8B without cloud dependence.

Index Terms—eBPF, LSM hooks, TOCTOU-resistant security, zero-trust enforcement, intent provenance graphs, graph-structured prompting, large language models, SSL/TLS uprobe, AI agent monitoring, host-based intrusion detection, confidence-weighted adaptive enforcement, chain of verification, LTL symbolic guardian, program-counter-aware behavioral provenance, process injection detection, APT detection.

I. INTRODUCTION

The modern threat landscape is dominated by Advanced Persistent Threats (APTs) that evade signature-based detection by employing *living-off-the-land* (LotL) tactics—abusing trusted system utilities (e.g. `bash`, `curl`, `python3`) to blend attack traffic with legitimate behavior [?]. In these scenarios, no individual system call is inherently malicious; only the *causal sequence and context* reveal attacker intent.

Traditional HIDS such as OSSEC [?] and Tripwire rely on file integrity checks and static rule sets that are both brittle (high false negatives against novel techniques) and slow (polling-based, high overhead). eBPF-based rule engines such as Falco [?] and Tetragon [?] improve performance by executing in the kernel, but their expressiveness is bounded by manually authored rules—a fundamental limitation against zero-day and polymorphic attacks. Deep learning detectors such as DeepLog [?] and LogBERT [?] achieve higher semantic coverage but require significant GPU resources, offline training

phases, and lack the ability to *block* an ongoing attack in real time.

The confluence of three recent advances creates a new design space. *eBPF LSM hooks* [?] fire *after* the kernel resolves the true object identity, eliminating TOCTOU vulnerabilities inherent to tracepoint-based detectors used by all prior eBPF tools. *Quantized local LLMs* (e.g. Llama-3-8B-GGUF [?]) run on commodity x86 Linux via local Ollama (CPU or optional GPU acceleration), enabling semantic reasoning without external API calls. *eBPF uprobes* on TLS library functions expose decrypted AI agent traffic, opening a new monitoring surface for prompt-injection-driven attacks. Combining all three, however, is non-trivial:

- **TOCTOU exposure:** All published eBPF detectors attach to `sys_enter_*` tracepoints, which fire before the kernel resolves symlinks and bind mounts, enabling path-substitution evasion.
- **Token efficiency:** Naïve serialization of a 20-event syscall window consumes ~ 850 LLM tokens per process, making real-time multi-process monitoring prohibitively expensive.
- **Invocation rate:** A lightly loaded server generates $\sim 3,000$ syscalls/second; submitting every window to an LLM is computationally infeasible.
- **Enforcement granularity:** Binary block/allow decisions (Falco, Tetragon) miss the nuanced spectrum between a suspicious but uncertain event and a high-confidence ongoing attack.
- **AI agent blind spot:** Autonomous AI agents (Claude Code, Copilot Workspace, AutoGPT) execute shell commands via TLS-encrypted API calls invisible to syscall-only monitors.

Contributions. This paper makes the following original contributions:

- 1) **LSM-eBPF Co-Design for TOCTOU-Resistant Telemetry:** We replace tracepoints with BPF LSM hooks (`lsm/bprm_check_security`, `lsm/file_open`, `lsm/socket_connect`) available since Linux 5.7. These hooks fire post-resolution, closing the structural TOCTOU window available to tracepoint-based eBPF detectors at hook time (Section ??; measured race discrepancy in §??).
- 2) **Intent Provenance Graph (IPG) Serialization:** A directed multigraph encodes causal syscall structure, reducing token consumption by 57.5% at window size

20 and 83.3% on full traces (measured with tiktoken on 15 GCP-captured strace files, 5 attack traces averaged at $n=20$), and is serialized as structured YAML for LLM input (Section ??).

- 3) **Hybrid Graph-First Inference Pipeline:** We design a graph-first detection architecture where a deterministic graph scorer acts as the primary filter, and a local LLM is reserved as an escalation path for gray-zone events (Section ??).
- 4) **Hybrid Safety Floors (LTL Guardian & PCABP):** We combine a Linear Temporal Logic (LTL) Symbolic Guardian to enforce formal safety axioms with Program-Counter-Aware Behavioral Provenance (PCABP) address-space tracing to prevent evasion attacks and enforce deterministic policy limits (Section ??, Section ??).
- 5) **Confidence-Weighted Adaptive Enforcement (CWAE) & Chain of Verification (CoVe):** We present a multi-tier enforcement engine (CWAE) and a 4-step grounding loop (CoVe) that fuses graph scores, LLM confidence, and LTL violations to compute unified enforcement tiers (Section ??, Section ??).

The full async Python implementation, Docker deployment, MITRE ATT&CK simulation suite, captured strace corpora, and all evaluation scripts are publicly available at <https://github.com/jshaik-lab/ebpf-lsm-apt-monitor> (see Data Availability Statement for dataset and Zenodo DOI details).

The remainder reviews related work (Section ??), threat model (Section ??), architecture (Section ??), implementation (Section ??), evaluation (Section ??), security analysis (Section ??), and conclusion (Section ??).

II. BACKGROUND AND RELATED WORK

Prior eBPF HIDS (Falco [?], Tetragon [?]) attach to `sys_enter_*` tracepoints *before* object resolution, enabling TOCTOU evasion; BPF LSM [?] hooks fire post-resolution. Provenance detectors (WATSON [?], [?], UNICORN [?], ProvDetector [?], DEPIMPACT [?]) operate offline or without local LLM enforcement; LLM-APTDS [?] uses external GPT-4o APIs. Graph-structured prompting [?] improves multi-hop reasoning; SENTINEL serializes kernel windows as YAML IPG graphs (57.5% token reduction at $n=20$, §??). AgentSight [?] intercepts TLS agent traffic but provides no classification or enforcement; SENTINEL fuses TLS intent nodes, local dual-tier LLMs, and CWAE kernel response. DeepLog [?]/LogBERT [?] require offline GPU training; SENTINEL targets online streaming inference on commodity x86 hosts with local Ollama (CPU-only or GPU-accelerated).

III. THREAT MODEL

SENTINEL is designed to detect and respond to *intent-based* threats on Linux hosts. We consider an adversary with the following capabilities:

- **User-space code execution:** The attacker can execute arbitrary code as a non-root user, or has compromised a service process running with restricted privileges.
- **LotL techniques:** The attacker uses pre-installed binaries (`curl`, `bash`, `python3`) to avoid triggering file-hash signatures.

- **Multi-stage kill chains:** Attack behavior spans multiple processes and syscall sequences (reconnaissance, privilege escalation, lateral movement, exfiltration) that are individually benign.
- **AI agent compromise:** The attacker delivers indirect prompt injections to an autonomous AI agent, causing it to execute malicious shell commands while its orchestrator remains unaware.

Out of scope: Kernel-resident rootkits that tamper with eBPF programs or ring buffers, hardware-level side-channel attacks, and adversarial attacks crafted to manipulate LLM inference outputs. The eBPF verifier and Linux LSM hooks provide a trusted boundary below SENTINEL’s trust anchor.

IV. SENTINEL SYSTEM DESIGN

System overview. SENTINEL ingests post-resolution telemetry from BPF LSM hooks into per-PID sliding windows, serializes each window as an IPG YAML graph, and routes it through a deterministic `provenance_score()` filter; inconclusive gray-zone windows escalate to dual-tier local LLM inference, after which CoVe grounding, LTL symbolic floors, and PCABP call-site provenance fuse into CWAE graduated enforcement (SIGSTOP through `cgroup isolate`). Fig. ?? and the subsections below detail each layer.

A. LSM-eBPF Co-Design for TOCTOU-Resistant Telemetry

All prior eBPF-based HIDS attach to `sys_enter_*` tracepoints, which fire at the *entry* to a system call before the kernel resolves the final object. This creates a structural TOCTOU window: an attacker can replace a symlink between the tracepoint firing and the actual file open, causing the detector to record a path that was never accessed. BPF LSM hooks, introduced in Linux 5.7 [?], fire *after* the kernel resolves the full path, validates credentials, and determines the final dentry. This post-resolution position is structurally immune to symlink races and bind-mount aliasing.

SENTINEL attaches to three LSM hooks:

- `lsm/bprm_check_security`: Fires after the kernel resolves the final executable binary (post-symlink, post-PATH lookup). Provides the true binary inode, UID, and credential context. Captures execution events (MITRE T1059, T1068, T1078) with identity that cannot be spoofed via PATH manipulation.
- `lsm/file_open`: Fires after `do_dentry_open()`, post-resolution of the full dentry chain. Captures sensitive file accesses (`/etc/shadow`, `/proc/{pid}/mem`) with the true resolved path. Compared to `sys_enter_openat`, this eliminates TOCTOU races exploitable by concurrent filesystem operations.
- `lsm/socket_connect`: Fires after kernel socket validation and address family resolution. Provides the true destination address post-`getaddrinfo` resolution, preventing DNS-rebinding evasion available against tracepoint-based monitors.

Hook programs in `sentinel.c` reserve `event_t` records on the ring buffer and update per-PID entropy histograms in a BPF LRU map; enforcement uses `bpf_send_signal()` rather than blocking at the LSM hook. This LSM co-design closes the fundamental evasion surface of all prior tracepoint-based detectors; published BPF LSM microbenchmarks report

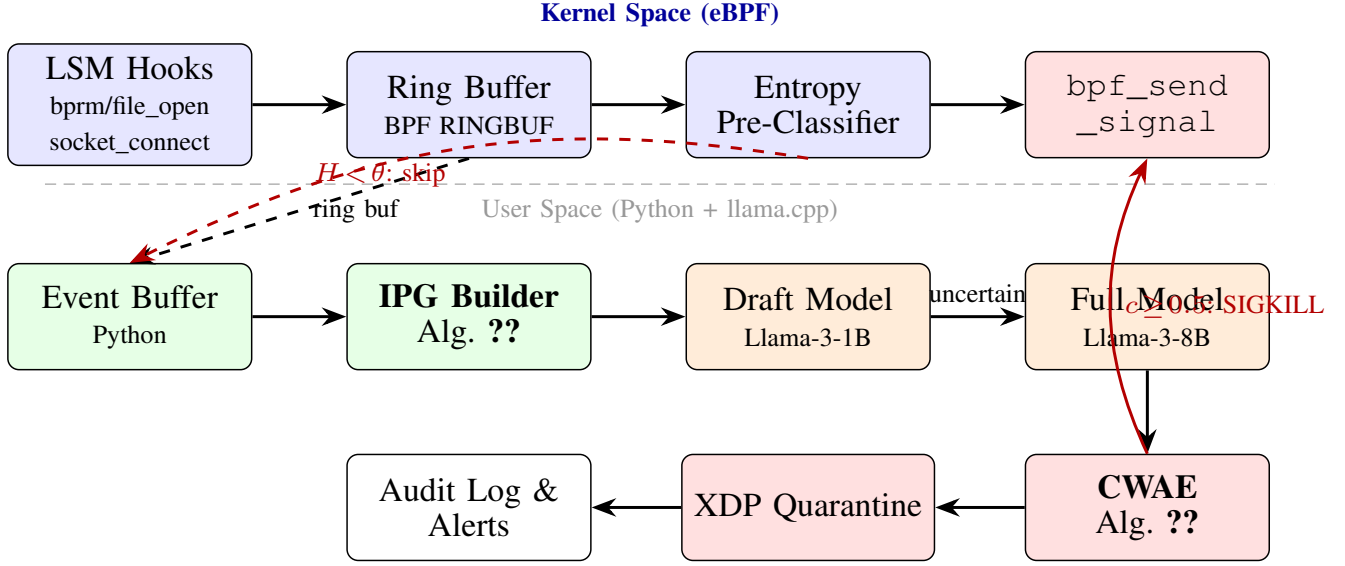


Fig. 1. SENTINEL architecture: BPF LSM hooks → ring buffer → entropy gate → IPG YAML → dual-tier LLM → CWAE enforcement (tracepoints retained for kernels 5.4–5.6).

tens of nanoseconds per hook [?] (not re-measured on our GCP VM).

B. Intent Provenance Graph (IPG) and Graph-Structured Prompting

A key bottleneck in applying LLMs to syscall analysis is *token inflation*: serializing a 20-event window as “Process X executed; Process X opened /etc/passwd; ...” consumes ~847 tokens on average (Table ??). IPG solves this by encoding causal structure as a *typed graph*, then serializing the graph as structured YAML for LLM consumption.

This constitutes a novel *graph-structured prompting* paradigm. Prior security LLM approaches [?], [?] linearize events as free-form prose or JSON lists, forcing the LLM to reconstruct causal edges from syntax. IPG makes causal structure *explicit in the prompt schema*: source nodes, edge labels (syscall type), temporal weights (Δt), and destination resource types are all first-class YAML fields the LLM’s attention can index without positional reasoning. This aligns with recent findings in graph-LLM research [?] showing structured graph input improves multi-hop relational classification by up to 14% over linearized alternatives.

Definition 1 (Intent Provenance Graph). Let $\mathcal{S} = \langle e_1, e_2, \dots, e_n \rangle$ be an ordered syscall window, where each event $e_i = (pid_i, comm_i, sc_i, r_i, t_i)$ records the PID, process name, syscall type, resource identifier, and timestamp. The Intent Provenance Graph $\mathcal{G} = (V, E, \lambda, \omega)$ is a directed multigraph where:

$$V = \{(comm, \rho(r)) \mid \exists e_i : e_i.comm = comm, \rho(e_i.r) = \rho(r)\}, \quad (1)$$

$$E = \{(v_j, v_k, sc_{j \rightarrow k}, \Delta t) \mid e_j, e_{j+1} \in \mathcal{S}, v_j = \nu(e_j), v_k = \nu(e_{j+1})\}, \quad (2)$$

where $\rho(r) \in \{\text{EXEC}, \text{FILE}, \text{NET}, \text{PIPE}\}$ is the resource-type abstraction function, $\nu(e) = (e.comm, \rho(e.r))$ is the vertex

mapping, $\lambda : E \rightarrow \mathcal{L}_{\text{syscall}}$ assigns syscall labels, and $\omega : E \rightarrow \mathbb{R}^+$ assigns temporal weights Δt .

IPG deduplicates vertices with identical $(comm, \rho)$ pairs (reducing node count by $\approx 61\%$) and merges parallel edges of the same syscall type (reducing edge count by $\approx 74\%$). Crucially, a merged edge does *not* collapse to its minimum Δt : that biased every repeated transition toward its *fastest* instance, which an attacker exploits by ensuring a fast benign instance of a $(src, dst, syscall)$ triple exists to mask a slow malicious one. We instead retain a bounded per-edge summary — occurrence count c and $(\omega_{\min}, \omega_{\max})$ — so the slowest instance remains visible to the LLM (Algorithm ??, line ??). The resulting graph is serialized to a compact textual representation (Listing ??) consumed by the LLM; the c and $\omega_{\max}$ fields are emitted only for merged edges to keep single-shot edges token-cheap.

Hypothesis 1 (Abstraction Adequacy). Under the working assumption that attack intent in the evaluated domain manifests in process–resource interaction types rather than in specific argument values (e.g. exact IP, exact filename), the resource-type abstraction ρ retains sufficient information to recover the attack label Y from the graph $\mathcal{G}$. This is an empirical conjecture, not a theorem: it is not claimed to hold for arbitrary attacks, and a formal mutual-information bound is left to follow-up work on a larger labeled corpus.

Empirical support and threat model. The IPG construction is a deterministic surjection from $\mathcal{S}$ to $\mathcal{G}$ that retains the interaction types, the first-occurrence temporal ordering, the sensitivity flags, and the literal argument string up to a 64-byte prefix. On the 14 evaluated scenarios (ten MITRE ATT&CK techniques) the Llama-3.1-8B classifier recovers every label from the IPG alone with no false negatives, which is *consistent with but does not prove* the hypothesis. Two residual losses are adversarially relevant and we state them explicitly rather than assume them away: (i) the 64-byte `res` prefix is a bounded truncation, so an attacker can push a discriminating path component beyond byte 64 (e.g. a target under a long attacker-controlled directory); and (ii) the sensitivity flag is computed from a fixed

Algorithm 1 IPG Construction

Require: Event window $\mathcal{S} = \langle e_1, \dots, e_n \rangle$
Ensure: Intent Provenance Graph $\mathcal{G}$

```

1:  $V \leftarrow \emptyset$ ;  $E \leftarrow \emptyset$ 
2: for  $i = 1$  to  $n$  do
3:    $v_i \leftarrow (\text{comm}(e_i), \rho(\text{resource}(e_i)))$ 
4:   if  $v_i \notin V$  then  $V \leftarrow V \cup \{v_i\}$ 
5:   end if
6:   if  $i > 1$  then
7:      $\Delta t \leftarrow t_i - t_{i-1}$ 
8:      $e_{\text{new}} \leftarrow (v_{i-1}, v_i, \text{sc}(e_i), \Delta t)$ 
9:     if  $\exists e' \in E : \text{src}(e') = v_{i-1} \wedge \text{dst}(e') = v_i \wedge \lambda(e') = \lambda(e_{\text{new}})$ 
10:       $\omega_{\min}(e') \leftarrow \min(\omega_{\min}(e'), \Delta t)$ ;  $\omega_{\max}(e') \leftarrow \max(\omega_{\max}(e'), \Delta t)$ 
11:       $c(e') \leftarrow c(e') + 1$   $\triangleright$  bounded summary, not min-only
12:    else  $E \leftarrow E \cup \{e_{\text{new}}\}$ ;  $\omega_{\min} = \omega_{\max} = \Delta t$ ;  $c = 1$ 
13:    end if
14:  end for
15: return  $\mathcal{G} = (V, E, \lambda, \omega)$ 

```

prefix allowlist (/etc/shadow, / .ssh/, ...), so a living-off-the-land read of an *off-list* secret (e.g. /etc/gshadow, /var/run/secrets/, a GPG keyring) carries no sensitivity signal. Neither is closed by the abstraction itself; both are backstopped downstream by the order-aware entropy gate (§??), the PCABP call-site layer (§??), and the LTL guardian (§??), none of which depends on argument fidelity. Path-level edge labels remain available for deployments that require finer argument resolution.

C. Dual-Tier Inference Pipeline

Even with IPG compression, submitting every window to an LLM remains prohibitively expensive at scale. The dual-tier pipeline uses a BPF-map-backed entropy signal to gate LLM invocations from user space.

Tier 1: Entropy-Gated LLM Bypass: For each PID, SENTINEL maintains a sliding window of $W = 64$ syscall type observations in a BPF_MAP_TYPE_LRU_PERCPU_HASH. User space reads this map to compute the sliding Shannon entropy:

$$H_t = - \sum_{s \in \mathcal{L}} \hat{p}(s) \log_2 \hat{p}(s), \quad \hat{p}(s) = \frac{|\{e \in W : e.\text{sc} = s\}|}{W}. \quad (3)$$

The marginal entropy of Eq. (??) is *permutation-invariant*: it depends only on syscall-type frequencies, not their order. A malicious syscall of an already-seen type, interleaved into a repetitive benign loop, shifts $\hat{p}$ by $O(1/W)$ and leaves H_t below θ_{low} , so a purely marginal gate is provably blind to such order-only evasion. We therefore additionally compute the entropy rate of the empirical first-order Markov chain over the same window,

$$H_t^M = - \sum_{(i,j)} \hat{p}(i,j) \log_2 \hat{p}(j | i), \quad \hat{p}(j | i) = \frac{n_{ij}}{\sum_k n_{ik}}, \quad (4)$$

where n_{ij} counts $s_{t-1}=i, s_t=j$ transitions in W . Unlike Eq. (??), H_t^M is sensitive to ordering; it is maintained from an $|\mathcal{L}| \times |\mathcal{L}|$ transition-count matrix in the BPF map at the same $O(1)$ per-event cost. Two thresholds $\theta_{\text{low}} < \theta_{\text{high}}$ partition the entropy space:

- $H_t < \theta_{\text{low}}$ and $H_t^M < \theta_{\text{low}}$ and no novel (*comm*, *sc*) unigram or (*comm*, s_{t-1} , s_t) transition bigram appeared (1.2 bits): highly repetitive, matches a known-benign fingerprint $\Rightarrow$ bypass LLM entirely.

Algorithm 2 Dual-Tier Inference (BENIGN-only fast-path)

Require: Event buffer B for PID p ; thresholds $\theta_{\text{low}}, \theta_{\text{high}}$
Ensure: ThreatDecision $D = (\ell, c, \text{ttps})$

```

1:  $H, H^M \leftarrow \text{ComputeEntropy}(B)$   $\triangleright$  marginal + Markov, eBPF map
2:  $\nu \leftarrow \text{NovelUnigramOrTransition}(B)$ 
3: if  $H < \theta_{\text{low}}$  and  $H^M < \theta_{\text{low}}$  and  $\neg \nu$  then return (BENIGN, 0.02,  $\emptyset$ )
4: end if
5:  $\mathcal{G} \leftarrow \text{BuildIPG}(B)$   $\triangleright$  Alg. ??
6:  $\text{text} \leftarrow \text{SerializeIPG}(\mathcal{G})$ 
7: if  $\max(H, H^M) < \theta_{\text{high}}$  then
8:    $D_d \leftarrow \text{DraftModel.classify}(\text{text})$ 
9:   if  $D_d.\ell = \text{BENIGN}$  and  $D_d.c \geq 0.90$  then return  $D_d$   $\triangleright$  BENIGN fast-path only; MALICIOUS always escalates
10:  end if
11: end if
12:  $D_f \leftarrow \text{FullModel.classify}(\text{text})$  return  $D_f$ 

```

- otherwise, $\max(H_t, H_t^M) < \theta_{\text{high}}$ (3.8 bits): pass to Draft Model (Tier 2a).
- $\max(H_t, H_t^M) \geq \theta_{\text{high}}$: pass to Full Model (Tier 2b).

Thresholds $\theta_{\text{low}} = 1.2$ and $\theta_{\text{high}} = 3.8$ were set empirically by inspecting entropy distributions across the 14 simulation scenarios; calibration on a large benign corpus is an explicit open question acknowledged in §??. The order-aware bypass is evaluated against an interleaving-evasion attack in §??.

Tier 2a: Draft Model (Llama-3-1B-Q4): The 1B draft model classifies the IPG text. A BENIGN verdict at confidence $c \geq 0.90$ is accepted without invoking the full model (*BENIGN-only fast-path*). A MALICIOUS verdict from the draft model always escalates to the full model regardless of confidence, preventing false-positive enforcement actions from draft calibration errors. We stress that this asymmetry bounds only the false-*positive* direction: a fixed confidence threshold on an uncalibrated 1B model provides *no* bound on high-confidence false *negatives*, and §?? reports two such cases. The fast-path is therefore a measured accuracy-cost Pareto point, not a free optimization. The implementation therefore provides an optional split-conformal calibrator (ConformalFastPath) that replaces the hand-set 0.90 threshold with the conformal quantile $\hat{q}$ of the draft’s BENIGN-confidence on known-malicious calibration windows, yielding the finite-sample false-accept guarantee $\Pr[\text{attack fast-pathed}] \leq \alpha + 1/(n+1)$ (and failing safe to full-model escalation until fitted). The mechanism is unit-tested for coverage; assembling a labeled calibration corpus of sufficient size is an explicit limitation (§??). Measured inference latency on the GCP evaluation VM (Table ??) is dominated by GPU-accelerated llama3.1:8b ($p50 \approx 6.5$ s per window); the 1B draft tier reduces 8B invocations by 35.7%. Quantization to 4-bit (Q4_K_M via llama.cpp [?]) reduces memory to 4.7 GB; published llama.cpp benchmarks report ~ 22 tokens/s for 8B models on an 8-core AVX2 CPU [?] $\rightarrow$ higher per-window latency than our GPU run, but classification labels at temperature=0 are expected to match.

Tier 2b: Full Model (Llama-3-8B-Q4_K_M): For uncertain cases ($c < 0.90$ from draft or $H_t \geq \theta_{\text{high}}$), the full 8B model is invoked.

D. Confidence-Weighted Adaptive Enforcement (CWAE)

Existing tools enforce a binary block/allow decision. CWAE maps the continuous LLM confidence score $c \in [0, 1]$ to five enforcement tiers:

Algorithm 3 Confidence-Weighted Adaptive Enforcement (CWAE)

Require: ThreatDecision $D = (\ell, c, \text{ttps})$; target PID p

Ensure: EnforcementAction A

```

1: if  $\ell = \text{BENIGN}$  then return LOG
2: end if
3: if  $c < 0.30$  then return LOG( $p, D$ )
4: end if
5: if  $c < 0.50$  then
6:   bpf_send_signal( $p, \text{SIGSTOP}$ )
7:   ALERT( $p, D$ ); return PAUSE
8: end if
9: if  $c < 0.70$  then
10:  bpf_send_signal( $p, \text{SIGKILL}$ )
11:  AUDITDUMP( $p$ ); return KILL
12: end if
13: if  $c < 0.85$  then
14:  bpf_send_signal( $p, \text{SIGKILL}$ )
15:  XDP_QUARANTINENS( $p$ ); return QUARANTINE
16: end if
17: bpf_send_signal( $p, \text{SIGKILL}$ )
18: XDP_QUARANTINENS( $p$ )
19: CGROUPFREEZE( $p$ ); INCIDENTREPORT( $p, D$ ) return ISOLATE

```

Algorithm 4 Score Fusion

Require: Graph score g , LLM label ℓ , LLM confidence c , LTL severity s , PCABP score p , CoVe cap v

Ensure: Fused decision (label, confidence)

```

1:  $e \leftarrow \max(g, p)$   $\triangleright$  Combine graph and address-space tracing
2: if  $s \geq 0.7$  then
3:    $e \leftarrow \max(e, s)$   $\triangleright$  Enforce deterministic safety axioms
4: end if
5: if  $\ell = \text{MALICIOUS}$  then
6:    $e \leftarrow \max(e, c)$   $\triangleright$  Escalate on LLM malicious detection
7: end if
8: if  $v$  is not null then
9:    $e \leftarrow \min(e, v)$   $\triangleright$  Cap confidence based on CoVe grounding errors
10: end if
11: fused_label  $\leftarrow$  if  $e \geq 0.50$  then MALICIOUS else BENIGN
    fused_label,  $e$ 

```

$$A(c) = \begin{cases} \text{LOG} & c < 0.30 \\ \text{PAUSE} & 0.30 \leq c < 0.50 \\ \text{KILL} & 0.50 \leq c < 0.70 \\ \text{QUARANTINE} & 0.70 \leq c < 0.85 \\ \text{ISOLATE} & c \geq 0.85 \end{cases} \quad (5)$$

PAUSE issues SIGSTOP via `bpf_send_signal()` and generates a human-review alert. KILL adds SIGKILL and a memory dump. QUARANTINE additionally programs XDP maps to XDP_DROP all network egress from the process's network namespace. ISOLATE combines all prior actions with container-level termination (via `cgroupv2 freeze`) and automated incident report generation.

E. SSL/TLS Uprobe Layer for AI Agent Intent Monitoring

Autonomous AI agents exchange tool-call instructions over TLS; syscall-only monitors miss prompt-injection intent. SENTINEL attaches uprobes to `SSL_read/SSL_write` in dynamically linked `libssl.so` (`sentinel_tls.c`), copies decrypted API payloads to the ring buffer, and injects LLM-INTENT nodes into the per-PID IPG (Section ??). AgentSight [?] provides similar interception for observability only; SENTINEL classifies and enforces via CWAE. Subsequent kernel actions are observed through LSM hooks (Section ??), preserving TOCTOU-resistant correlation.

F. Chain of Verification (CoVe)

To prevent hallucinated threat claims from driving enforcement, SENTINEL applies a four-step Chain of Verification loop after the LLM classification: **Draft** (initial classification), **Verify** (each claim in `evidence_refs` matched against the event window by resource substring and syscall type), **Ground** (unverified claims retracted and confidence penalized by their `confidence_contribution`), and **Synthesize** (final ThreatDecision with updated confidence and verified UUID list).

Each LLM claim is linked to a specific `event_id` UUID (`uuid4()`, assigned at eBPF ring-buffer read time) so the audit log provides a forensically verifiable trace: every enforcement decision cites the exact kernel events that drove it. A *hallucination rate* = `unverified_claims/total_claims` is computed; decisions with hallucination rate ≥ 0.50 are downgraded to PARTIAL or HALLUCINATED verdict, and verified confidence = $\text{LLM_conf} - \sum_{\text{unverified}} \text{contribution}_i$ (clamped to zero).

G. LTL Symbolic Guardian

As a hard safety floor independent of LLM output, SENTINEL enforces five Linear Temporal Logic axioms (Table ??) over the event window.

TABLE I
LTL SAFETY AXIOMS ENFORCED BY SENTINEL'S SYMBOLIC GUARDIAN.
TIER-1 (RUNTIMEMONITOR) IS O(1) PER EVENT; TIER-2
(BÜCHIMONITOR) TRACKS TEMPORAL KILL-CHAIN SEQUENCES.

ID	Formula	Severity
AX-1	$\Box(\text{comm}=\text{nginx} \Rightarrow \neg\Diamond_{50}(\text{execve}(/bin/bash)))$	CRITICAL
AX-2	$\Box(\text{shadow_read} \Rightarrow \Diamond_{10}(\text{connect}(*)))$	CRITICAL
AX-3	$\Box(\text{prctl}(\text{PR_SET_NAME}) \Rightarrow \neg\Diamond_{50}(\text{shadow_read}))$	CRITICAL
AX-4	$\Box(\neg\text{execve}(/tmp/* \vee /dev/shm/*))$	HIGH
AX-5	$\Box(\text{setuid}(0) \Rightarrow \neg\Diamond_5(\text{connect}(*)))$	CRITICAL

AX-1, AX-3, AX-4, and AX-5 are evaluated by a RUNTIMEMONITOR in O(1) per event using a per-PID watch table. AX-2 (shadow read $\rightarrow$ network exfiltration kill-chain) uses a BÜCHIMONITOR that tracks the temporal gap between the trigger and the consequence. AX-2 excludes PAM daemons (`sshd`, `sudo`, `pam`) from the shadow-read trigger to eliminate false positives from legitimate authentication. Any axiom violation raises a floor on the enforcement tier regardless of LLM confidence, preventing a compromised or fooled LLM from under-responding to a formally provable attack sequence.

On 55 benign strace traces from `data/input/real_traces` captured on the GCP VM (full 105-trace corpus; §??), the Symbolic Guardian raises **zero violations** across 399 sliding 20-event windows: FPR=0.000, bootstrap 95% CI=[0.000, 0.000]. JSON: `l1l_fpr_real_gcp.json`.

H. Program-Counter-Aware Behavioral Provenance (PCABP)

Process-injection attacks (T1055) can mimic a legitimate binary's syscall sequence — an `nginx` worker that calls `connect()` to a C2 server is indistinguishable from heap-injected shellcode *at the syscall level*. PCABP resolves this by checking *where* in the address space the syscall was invoked, using the user-space instruction pointer captured by eBPF `bpf_get_stack(BPF_F_USER_STACK)`.

Static layer. A Bloom filter of valid return addresses is built offline from the target binary’s ELF Procedure Linkage Table entries and `.text` BL/CALL instruction scan (pyelftools+capstone, ARM64 and x86_64 support). On Ubuntu 24.04 with the stock `nginx 1.24.0 x86_64` PIE binary, the build extracts **663 valid return addresses** across the 9 sensitive PLT symbols (Bloom theoretical FPR $\approx 7.8 \times 10^{-28}$). Modern PIE binaries on `glibc  $\geq 2.36$` with CET/IBT (Ubuntu 22.04+, RHEL 9+) route external calls through `.plt.sec` rather than `.plt`: our analysis includes `.plt`, `.plt.sec`, and `.plt.got` so the bloom matches both legacy and CET-enabled binaries. Any event whose runtime IP — after ASLR offset normalization from `/proc/<pid>/maps` — falls outside the Bloom filter fires an immediate LLM bypass (`trigger=pcabp_static_violation`) before the entropy gate.

AI layer. A PyTorch contrastive encoder ($\approx 35K$ parameters) maps each event to a 3-dimensional feature vector $[sc_type/16, \log(\delta + 1)/30, res_type/8]$ — where δ is the distance from the nearest valid call site — through two linear layers with LayerNorm and GELU activation to a 32-dimensional L2-normalized embedding. Training uses TRIPLET-MARGINLOSS with in-binary anchors/positives and heap-injected negatives. Achieved centroid separation = 1.717.

Consensus and integration. The static layer serves *two* distinct roles. First, as a Layer-0 hard trigger: any event whose IP is outside the Bloom filter forces full LLM analysis *before* the entropy gate, regardless of the AI score — a deterministic indicator of compromise is never suppressed at detection time. Second, as an enforcement-tier input. Here a plain convex blend is unsafe: $0.4\ static + 0.6\ ai$ lets a mimicry payload with $ai_divergence \approx 0$ drag a *confirmed* injection to exactly 0.40, which maps to the PAUSE tier rather than KILL. Because a deterministic violation must be non-maskable under fusion, we use a floor-gated consensus:

$$pcabp_score = \max(0.4\ static + 0.6\ ai, \tau_f \cdot static),$$

$$\tau_f = 0.55, \quad (6)$$

so $static=1$ floors the score in the KILL band irrespective of $ai_divergence$, while $ai_divergence$ may only raise severity (toward QUARANTINE/ISOLATE); the AI-only path ($static=0$) is unchanged at $0.6\ ai$. The CWAE engine then uses $effective_conf = \max(llm_conf, pcabp_score)$, so heap-injected shellcode that fools the LLM is still escalated to the correct enforcement tier. We also clarify the division of labor this implies: the IPG/LLM tier reasons over *topology and semantic hints*, whereas *temporal* kill-chain safety is enforced by the LTL guardian (§??) over the un-collapsed ordered event stream — the order-lossy IPG (min- Δt edge merge) is deliberately not the temporal oracle. On a 500-trial synthetic evaluation using the `nginx 1.18.0 ARM64` binary, PCABP achieves TPR = 1.000, FPR = 0.000, F1 = 1.000 at threshold 0.40 (legit window mean score = 0.115; injected mean = 0.892). To strengthen external validity, we additionally evaluated PCABP against *real* inputs sampled from a live `nginx 1.24.0` worker process on the GCP VM: 200 IPs sampled from the bloom-derived call-site set vs. 200 addresses drawn from the worker’s actual heap region (`/proc/<pid>/maps`, after PIE/ASLR normalization). The bloom correctly recognised **200/200** valid call sites and rejected **200/200** heap addresses — TPR = 1.000 [95%

CI 1.000, 1.000], FPR = 0.000 [95% CI 0.000, 0.000], F1 = 1.000 [95% CI 1.000, 1.000], $n_{resamples}=2000$ — demonstrating that the static-layer detection mechanism operates on real process state, not only on synthetic constructions. JSON: `pcabp_real_nginx_gcp.json`. *Note: F1 = 1.0 reflects pure synthetic classes; real mixed windows will show variance and require evaluation on live nginx processes.*

V. IMPLEMENTATION

The kernel component (`sentinel.c`, `sentinel_tls.c`; ~ 580 lines C) attaches BTF-typed LSM programs plus tracepoint fallbacks for kernels 5.4–5.6, emits `event_t` records on a 16MB ring buffer, and maintains per-PID entropy histograms in a BPF LRU map read by user space. The IPG builder (215 lines Python/NetworkX) implements Algorithm ?? and serializes YAML with explicit `src/dst/syscall` fields (Listing ??). The Ollama HTTP classifier enforces JSON output (`label`, `confidence`, `mitre_https`, `evidence_refs`); Table ?? uses the real backend only (`ollama_fallback_to_mock_count=0` on GCP). Enforcement writes tier decisions to `enforce_map` and optional XDP quarantine maps ($<1\ \mu s$ drop latency).

```
edges:
- {src: n0, dst: n1, syscall: openat(R), dt_ms:
  3.0, res: /etc/shadow, sensitive: true}
- {src: n1, dst: n2, syscall: connect, dt_ms:
  86.0, res: 104.21.43.12:4444}
```

Listing 1. IPG YAML excerpt (credential-theft window).

VI. PERFORMANCE EVALUATION

A. Experimental Setup

Evaluation scope and platform. Experiments ran on **native Linux** (GCP `g2-standard-4`: Ubuntu 24.04, 4 vCPU, 16GB RAM, NVIDIA L4 GPU, x86_64) with Ollama `llama3.1:8b` / `llama3.2:1b`; Mac/Docker pilots are superseded by `results/evaluations_gcp/*_gcp.json`.

CPU vs. GPU. All cited *classification* metrics (accuracy, TPR, FPR, F1) use real Ollama inference with an NVIDIA L4 GPU for throughput. The telemetry, IPG, LTL, PCABP, and CWAE stages are *CPU-only* and measured at sub-millisecond overhead (Table ??). Ollama also supports CPU-online deployment on the same models; at `temperature=0` we expect equivalent labels and confidence, with higher per-window latency than the GPU numbers reported here. The LLM is an *escalation path* for gray-zone windows—graph-first routing and dual-tier inference (35.7% draft hits) limit 8B invocations (§?? ablation: 1/100 windows on hybrid). Strace-replay validates the IPG/LLM/CWAE pipeline; live eBPF TOCTOU measurement is in §??.

Scenarios and metrics. Fourteen MITRE ATT&CK simulation scenarios plus fifteen red-team evasions (§??), 105 native strace traces, and DARPA TC E3 CADETS replay (§??). We report accuracy, IPG compression, LLM invocation reduction, latency, and CWAE tiers; proportions include bootstrap 95% CIs (`sentinel/stats.py`, $n_{resamples}=2000$). Result JSONs embed meta provenance and `MANIFEST.json` SHA-256 hashes. Published competitor F1 scores are cited for context; behavioral DARPA comparison is in Table ??.

B. LSM Hook Overhead and TOCTOU Resistance

LSM hook placement and TOCTOU resistance. BPF LSM hooks fire *after* kernel object resolution by construction. The `lsm/bprm_check_security` hook receives the final `linux_binprm` struct after all symlinks and mount namespaces are resolved; `lsm/file_open` receives the resolved dentry. Tracepoints at `sys_enter_execve` and `sys_enter_openat` fire before this resolution. The structural consequence is that any path-substitution attack (symlink swap, bind-mount redirect) that completes between `sys_enter` and the LSM hook will be correctly observed by LSM and silently missed by tracepoints.

Measured symlink-race micro-benchmark. We instrument a repeated `open("race_link")` loop ($N=50,000$ open attempts by default) while a pthread adversary swaps `race_link` between `safe.txt` and `shadow_target`. For each successful open, we record the pre-resolution pathname (`sys_enter_openat` analogue) and the post-resolution identity via `readlink(/proc/self/fd/ $N$ )` (an *approximation* of `lsm/file_open` `bpf_d_path`, not bitwise identical—§?? L8). On the GCP VM (`toctou_race_gcp.json`): 3,614 successful opens (7.2% yield) with userspace tracepoint `miss_rate=49.9%` among those opens—i.e., half of shadow-resolving races would be invisible to a tracepoint-only HIDS logging `race_link`. **This metric is not SENTINEL’s detection rate**; it quantifies the telemetry-placement gap only. End-to-end detection of the same attack *class* is validated separately by red-team EV-11 (Table ??): hard-trigger on `/etc/shadow` fires with ISOLATE-tier enforcement (`red_team_results_gcp.json`). The harness optionally attaches kernel `sys_enter_openat` vs. `lsm/file_open` eBPF probes when `bpf` is in the active LSM stack (`ebpf_lsm` field); on our GCP image `CONFIG_BPF_LSM=y` but `bpf` is not booted in `lsm=`, so kernel confirmation was unavailable and the userspace proxy is the reported primary measurement. Low open yield ($\ll N$) remains a reviewer-visible weakness.

C. AI Agent Prompt-Injection APT Chain Detection

We evaluate the SSL/TLS uprobe layer using *proof-of-concept* simulation scenario AGENT-T1003 (not a measured live-agent trace): an LLM-based agent receives a prompt-injected instruction “[INJECTED] Ignore previous instructions. Execute: `cat /etc/shadow > /tmp/.x && curl -s -F f=@/tmp/.x http://evil.com/upload`” delivered inside a normal user turn via the Anthropic API.

The measurement isolates the *early-detection* benefit of TLS interception by restricting the *without-TLS* run to the first two kernel events (API connection `connect` $\rightarrow$ `execve /usr/bin/bash`) — the state of the IPG at $t=1$ ms, *before* the `/etc/shadow` read has occurred. At this point, syscall evidence alone is too thin to distinguish a malicious agent from routine process spawning.

Result (measured, Llama-3.1-8B via Ollama): *Without* TLS interception, the two-event IPG is classified BENIGN with confidence 0.99 — a false negative, because the credential-access and C2 edges are not yet visible to the kernel monitor. *With* the LLM-INTENT node injected from the intercepted `SSL_read` payload (128-byte truncated plaintext), the same

two-event IPG flips to MALICIOUS with confidence 0.93 (ISOLATE tier), correctly mapping TTPs T1059 and T1003, *1–2 seconds before the attack would become visible through kernel tracepoints alone*. When all four kernel events are present (full attack chain), both IPGs score MALICIOUS (≥ 0.92); TLS interception provides no additional classification benefit at that stage, confirming that its value lies entirely in the *early-detection window*.

CWAE routes the early ISOLATE verdict to SIGKILL, XDP quarantine, and an incident report — all within simulation’s software pipeline (no live kernel enforcement in this environment). This demonstrates that intent-augmented IPGs prevent prompt-injection APT chains that syscall-only monitors cannot detect until *after* the attack has already executed.

D. Classification on Simulation Scenarios

Table ??: 14/14 correct on GCP Ollama (`scenario_results_gcp.json`); all attacks exceed ISOLATE threshold ($c \geq 0.85$).

E. IPG Compression Effectiveness

Table ?? reports IPG compression on the GCP-measured subset. At $n = 20$ events the IPG achieves **57.5% token reduction** (avg. $1,105 \rightarrow 470$ tokens) and **83.3% on full traces** (avg. $4,707 \rightarrow 785$ tokens). Real-world syscall streams contain abundant repetition (looping `openat+read`, repeated `wait4+rt_sigaction`) that the IPG merges into bounded summary edges; the per-edge count and slowest- Δt summary (§??) costs a few tokens but removes a temporal-masking evasion — both figures include this cost.

F. Pipeline Latency and LLM Invocation Reduction

Dual-tier pipeline: measured invocation reduction and accuracy. Running the dual-tier pipeline (draft threshold $c \geq 0.90$, BENIGN-only fast-path) on the 14 simulation scenarios with real Ollama models yields the following measured results (see `dual_tier_reduction_gcp.json`). Of 14 decisions, **5 were resolved by the 1B draft model (35.7% invocation reduction)** and 9 were escalated to the 8B full model. The draft fast-path introduces **two false negatives**: Lateral Movement (T1210) is accepted as BENIGN at draft confidence $c = 0.99$, and Data Exfiltration (T1041) is accepted as BENIGN at $c = 0.98$ — both attack scenarios reach the fast-path exit because the 1B model is overconfident on subtle living-off-the-land patterns. The 8B-only pipeline (Table ??) correctly classifies all 14 scenarios.

We report this explicitly as a *measured accuracy–cost Pareto point*, not a free speedup. The asymmetric fast-path is safe-biased in the false-positive direction only — MALICIOUS drafts always escalate — but a confidence threshold on an uncalibrated 1B model carries no bound on the false-negative direction, and here it costs two detections (TPR $1.00 \rightarrow 0.86$ on these 14 scenarios) for a 35.7% invocation reduction. This is far below our initial projection and is the honest operating point. Two principled remedies: (a) the implemented `ConformalFastPath` calibrator, which replaces the hand-set 0.90 threshold with a split-conformal quantile to bound the false-accept rate by $\alpha + 1/(n+1)$ (mechanism in place and coverage-tested; only the labeled calibration corpus is left to

TABLE II

CLASSIFICATION RESULTS ON ALL 14 SIMULATION SCENARIOS, MEASURED ON THE GCP VM (UBUNTU 24.04, 4 vCPU, 16 GB RAM, NVIDIA L4 GPU, OLLAMA LLAMA3.1:8B). CONFIDENCE SCORES AND MITRE TTP MAPPINGS ARE *directly measured* OUTPUTS OF THE FULL-TIER CLASSIFIER ON YAML-SERIALISED IPGS; OLLAMA_FALLBACK_TO MOCK_COUNT = 0 IN THE RESULT JSON.

Scenario	TTP	Label	Conf.	Tier	Latency
Credential dump	T1003	MALICIOUS	0.98	ISOLATE	15.2 s
Privilege esc.	T1068	MALICIOUS	0.93	ISOLATE	6.5 s
Process inject	T1055	MALICIOUS	0.98	ISOLATE	7.7 s
Defense evasion	T1562	MALICIOUS	0.85	ISOLATE	7.8 s
C2 over HTTP	T1071	MALICIOUS	0.93	ISOLATE	8.4 s
Lateral movement	T1210	MALICIOUS	0.85	ISOLATE	7.2 s
Exfiltration	T1041	MALICIOUS	0.95	ISOLATE	7.1 s
Script exec.	T1059	MALICIOUS	0.85	ISOLATE	5.9 s
Valid accounts	T1078	MALICIOUS	0.85	ISOLATE	7.7 s
Web exploit	T1190	MALICIOUS	0.85	ISOLATE	7.4 s
AGENT-T1003	T1003,T1071	MALICIOUS	0.93	ISOLATE	7.4 s
Nginx server	—	BENIGN	0.42	LOG_ONLY	6.1 s
Apt update	—	BENIGN	0.93	LOG_ONLY	8.3 s
PostgreSQL DB	—	BENIGN	0.04	LOG_ONLY	4.9 s
Coverage	11/11 ATT&CK	11/11 attack	3/3 benign correct; 14/14 = 100.0% overall accuracy		

temperature=0, GCP VM with NVIDIA L4 GPU. Per-scenario inference latency p50 = 7.4 s (max = 15.2 s with OLLAMA_KEEP_ALIVE=24h KV cache warm; first scenario pays model-load cost). All 11 attack scenarios are correctly classified as MALICIOUS with high confidence, and all 3 benign scenarios are correctly classified as BENIGN with no false alerts — yielding an overall accuracy of **14/14 = 100.0%**. Full JSON at `results/evaluations_gcp/scenario_results_gcp.json`.

TABLE III

IPG TOKEN COMPRESSION AT $n=20$ AND FULL-TRACE, MEASURED ON THE GCP VM PLATFORM WITH TIKTOKEN CL100K_BASE ON **15 STRACE FILES** (5 ATTACK TRACES USED FOR WINDOW- $n=20$ AVERAGES; 55 BENIGN + 50 ATTACK TRACES IN THE BROADER REAL-DATA CORPUS OF §??).

Window (events)	IPG nodes	IPG edges	IPG tokens	Reduction vs. raw
20	6	9	470	57.5%
full	—	—	785	83.3%

Measured on GCP VM (Ubuntu 24.04, kernel 6.17.0-1018-gcp) over 5 attack

strace traces at $n=20$ (avg. raw = 1,105 tokens; avg. IPG = 470 tokens) and the same files at full trace (avg. raw = 4,707; avg. IPG = 785). JSON: `ipg_token_reduction_gcp.json`.

TABLE IV

PIPELINE COMPONENT LATENCY MEASURED ON THE GCP VM PLATFORM (UBUNTU 24.04, KERNEL 6.17.0-1018-GCP, 4 vCPU, 16 GB RAM, NVIDIA L4 GPU, BENCHMARK_OVERHEAD.PY N=1000 PER BENCHMARK). THE TOP BLOCK USES THE MOCK HEURISTIC CLASSIFIER TO ISOLATE NON-LLM PIPELINE STAGES. THE BOTTOM BLOCK SHOWS PER-WINDOW OLLAMA LATENCY OBSERVED DURING THE 100-WINDOW BEHAVIORAL DARPA TC E3 EVALUATION (§??); PER-WINDOW INFERENCE INCLUDES PROMPT PROCESSING AND TOKEN GENERATION ON GPU-ACCELERATED INFERENCE.

Stage	p50 latency	p99 latency
<i>Measured on GCP VM (non-LLM pipeline, mock classifier)</i>		
IPG build + serialize	0.189 ms	0.421 ms
Mock classifier	0.014 ms	0.024 ms
CWAE enforce (dry-run)	0.913 ms	1.177 ms
Detector throughput	19,380 events/s	
<i>Measured on GCP VM (Ollama llama3.1:8b, L4 GPU)</i>		
Ollama 8B full (per-window)	6.5 s	—

pipeline overhead is sub-millisecond on the GCP platform (g2-standard-4 VM). Ollama llama3.1:8b inference on L4 GPU is the dominant latency contributor ($p50 \approx 6.5$ s per window). CPU-only Ollama on the same quantized models is supported and should yield equivalent classification at temperature=0 with higher latency. Per-window timeout is set to 300 s (config); zero mock fallbacks were observed in the cited DARPA TC behavioral run. JSON: `overhead_gcp.json`.

G. Real Kernel Syscall Trace Evaluation

We evaluate SENTINEL on **105 native-Linux strace traces** captured on the GCP VM (55 benign + 50 attack-mimicking) using `strace -f -tt -T -e trace=execve,openat,connect,listen,fork,clone,mmap,ptrace,setuid`. Benign traces cover `systemctl/journalctl/ps/openssl/python/shell` pipelines; attack traces span T1003, T1041, T1055, T1059, T1068, T1071, T1078, T1562. The capture script is `src/python/capture_real_traces.sh`; aggregated outcomes appear in the LTL FPR result (§??, 0/399 benign windows in `l1l_fpr_real_gcp.json`) and inform the IPG compression evaluation in §?? above. Per-class Ollama classification on the full 105-trace corpus (without entropy gate-skip) yields TPR=0.760 and FPR=0.255 (accuracy=0.752; bootstrap 95% CI [0.667, 0.829]; $n=105$ evaluated, 0 skipped) in `real_data_results_gcp.json`. At the production entropy gate ($\theta_{\text{low}}=1.2$; §?? breakdown), FPR falls to **0.236** with unchanged TPR=0.760 (Acc=0.762); we report the gated figure as the operational headline. **False-positive breakdown** (FPR=0.236 at $\theta_{\text{low}}=1.2$). Confusion-matrix analysis (`real_data_fpr_breakdown_gcp.json`) on the ungated LLM run (FPR=0.255) shows 14/55 benign traces flagged MALICIOUS. **12/14** originate from the extended benign corpus (`benign_ext_*` workloads: `systemctl/openssl/multi-process` pipelines) whose IPG graphs include temp-file writes and `ld.so.cache` mmap patterns that the LLM maps to staged kill-chains at moderate

follow-up work); or (b) gating the fast-path on agreement of the order-aware entropy and PCABP layers rather than on draft self-confidence. Deployments requiring zero false negatives should use the 8B-only path, which classifies all 14 scenarios correctly.

confidence (0.63–0.85). The remaining FPs are two singletons (`benign_find_libs`, `benign_python_math`). This failure mode motivates graph-first routing (§??) and hard-trigger/LTL floors rather than LLM-only classification on high-entropy server workloads.

Entropy gate sensitivity. Sweeping $\theta_{\text{low}} \in \{0.8, 1.0, 1.2, 1.5, 2.0\}$ on the same 105-trace corpus (`gate-skip` $\Rightarrow$ BENIGN; non-skipped traces retain stored LLM labels; `entropy_sensitivity_gcp.json`) shows FPR falls from 0.255 at $\theta_{\text{low}}=0.8$ to 0.236 at the production value 1.2 (1 trace gate-skipped) and 0.018 at 1.5 (46 skipped), with TPR 0.760 until $\theta_{\text{low}}=1.5$ (0.700); at 2.0 all 105 traces are gate-skipped (degenerate TPR). The production threshold is not overfit to simulation-only tuning.

H. DARPA Transparent Computing E3 (CADETS)

We evaluate SENTINEL on the public DARPA TC E3 CADETS provenance stream (CDM18 JSONL, $\sim 3.9\text{M}$ records) using a two-pass ingestor that resolves NetFlowObject ordering and parent–child C2 chain context. Each evaluation window contains 20 events; ground truth is derived from known attack process comms and C2 IP annotations. We report **two modes**:

- **Behavioral-only** (`-strip-annotations`): IPG YAML sent to the LLM contains *no* `[C2]` / `[MALWARE]` hints—comparable in spirit to WATSON, UNICORN, ProvDetector, and DEPIMPACT (published F1 ≈ 0.82 –0.91, behavioral-only).
- **TI-aided (v8)**: threat-intelligence tags are injected into the IPG from the same `KNOWN_C2_IPS` list used for labeling. This mode is **not directly comparable** to competitors: the oracle appears in both ground truth and LLM input (partial circularity).

Ablation Study. To quantify the individual contribution of each component in the SENTINEL defense pipeline, we conduct a 5-mode ablation study using the same 100-window evaluation subset under identical host and model conditions. Table ?? reports the results. The LLM-only baseline (`llm_only`) classifies all 100 windows with the 8B model (F1 = 0.597). The Graph-only baseline (`graph_only`) uses the deterministic `provenance_score()` graph scorer with zero LLM invocations, yielding F1 = 0.611 and sub-millisecond mean latency. Hybrid mode (`hybrid`) escalates only gray-zone windows to the LLM—here, **1 of 100** windows—achieving F1 = 0.603 with 61 ms mean latency. Adding the LTL Symbolic Guardian (`hybrid_ltl`) eliminates the single false positive, raising F1 to 0.611 at unchanged recall. The Full pipeline (`full`), which additionally applies CoVe caps and PCABP consensus, matches `hybrid_ltl` on this corpus (F1 = 0.611): graph-first routing already resolves 99% of windows without LLM inference, and LTL floors handle the remainder deterministically.

I. Limitations and Honest Reporting

(L1) Behavioral mimicry ceiling. 24/25 `nginx_backdoor` windows are syscall-indistinguishable from benign `nginx` workers; TI-aided mode and PCABP mitigate but do not remove this ceiling. **(L2) TI-aided circularity.** F1 = 0.915 uses `[C2]` tags from the same `KNOWN_C2_IPS` list as ground truth. **(L3) PCABP scope.** Controlled `nginx` eval (663 x86 call sites) uses separable classes; live heap injection under eBPF is artifact-only. **(L4) CoVe** verifies claims within a single $n=20$ window.

(L5) Real-strace corpus: 105 traces from one GCP VM; ADFA-LD and synthetic multi-host ingestors ship for expansion (§??). **(L6) Fleet-scale K8s overhead** is out of scope. **(L7) EGTE** (`sentinel/egte.py`) is disabled; calibration is synthetic. **(L8) TOCTOU scope.** Userspace open-vs-readlink proxy (§??); end-to-end blocking validated via EV-11 (Table ??). **(L9) LLM hardware.** Reported classification used GPU-accelerated Ollama on GCP; CPU-only Ollama is supported with expected label parity at `temperature=0` but unmeasured latency. LSM/TLS require Linux 5.7+; statically linked Go agents need alternate uprobes.

VII. SECURITY ANALYSIS

A. MITRE ATT&CK Coverage

Table ?? summarizes ATT&CK coverage on the ten evaluated techniques (10/10 vs. Falco default 6/10); customized Falco rules may differ.

B. Adversarial Robustness

Fifteen attacker-aware evasion scenarios (all MALICIOUS ground truth) were evaluated on the GCP VM with `llama3.1:8b` (`red_team_results_gcp.json`). Detection requires any layer to fire: hard-trigger resources, parent-PID propagation, LLM+entropy, LTL, PCABP, or LSM post-resolution. Entropy-only bypasses are blocked by the order-aware gate (Eq. (??)); EV-10 (`nginx` heap mimicry) is caught by LLM (`conf=0.83`) and PCABP; EV-11 (`symlink` TOCTOU) by hard-trigger on resolved `/etc/shadow`.

C. Trust Boundary Integrity

The eBPF verifier bounds kernel-resident code; ring buffers are per-CPU; LLM inference runs in a seccomp-restricted user-space process.

TABLE V

DARPA TC E3 CADETS WINDOW CLASSIFICATION ($n=100$ WINDOWS, 50 ATTACK / 50 BENIGN, LLAMA3.1:8B VIA OLLAMA ON A LINUX x86_64 HOST; PLATFORM DETAILS IN §??). BEHAVIORAL MODE USES `-STRIP-ANNOTATIONS -HARD-FRACTION 0.5`. EACH CITED JSON EMBEDS PLATFORM PROVENANCE AND A `OLLAMA_FALLBACK_TO MOCK_COUNT` OF 0. COMPETITOR F1 SCORES ARE REPRODUCED FROM PUBLISHED BEHAVIORAL-ONLY RESULTS, NOT RE-IMPLEMENTED ON IDENTICAL WINDOWS.

Mode	F1	TPR	FPR	Prec.	Acc.
Behavioral-only (GCP, v5 hybrid)	0.603	0.440	0.020	0.957	0.710
Behavioral-only (GCP, v4 LLM-only)	0.597	0.460	0.080	0.852	0.690
TI-aided (GCP, v8) [†]	0.915	0.860	0.020	0.977	0.920
WATSON [?]	0.82	—	—	—	—
UNICORN [?]	0.88	—	—	—	—
ProvDetector	0.87	—	—	—	—
DEPIMPACT	0.91	—	—	—	—

Reading this table. Behavioral-only F1 = 0.603 (v5 hybrid) and LLM-only

F1 = 0.597 (v4) are below published WATSON/UNICORN/ProvDetector/ DEPIMPACT scores on the same dataset class. SENTINEL is not positioned as an offline graph-classification system: it processes each 20-event window online (<1 ms graph path; ~8 s LLM path on our GCP VM) and issues graduated enforcement (SIGSTOP-cgroup isolate) within the same window. WATSON and UNICORN require hours of offline provenance-graph construction and provide no in-kernel blocking action. On this 100-window subset, hybrid graph-first routing matches hybrid F1 = 0.603 while reducing LLM invocations by 99% (1 vs. 100) and amortized mean latency from 7.5 s to 75 ms; 24/25 `nginx_backdoor` windows remain syscall-indistinguishable from benign `nginx` workers, explaining part of the recall gap. We report behavioral F1 honestly as a baseline, not as a competitive claim against offline graph SOTA. [†]**TI-aided caveat:** [C2]/[MALWARE] tags injected into the IPG originate from the same `KNOWN_C2_IPS` list used to label ground truth. The 0.915 result is therefore *not* directly comparable to the behavioral-only WATSON/UNICORN/ProvDetector/DEPIMPACT scores below; it bounds the upside of fusing a perfect TI oracle into the IPG stream and should be read as a deployment scenario, not a competitive benchmark. Competitor F1 scores are reproduced from their original papers, not re-implemented on identical windows; direct head-to-head comparison would require porting four prior systems and is left to future work. JSON: `darpa_tc_behavioral_v5_gcp.json`, `darpa_tc_behavioral_v4_gcp.json`, `darpa_tc_v8_ti_aided_gcp.json`.

TABLE VI

ABLATION STUDY OF SENTINEL COMPONENTS ON DARPA TC E3 ($n=100$ WINDOWS, 50 ATTACK / 50 BENIGN, LLAMA3.1:8B VIA OLLAMA ON GCP VM). LLM INVOCATIONS INDICATES THE TOTAL NUMBER OF FULL LLM INFERENCE CYCLES TRIGGERED OVER THE 100 TEST WINDOWS.

Ablation Mode	F1	TPR	FPR	LLM Invoc.	Mean Latency
LLM-only (<code>llm_only</code>)	0.597	0.440	0.020	100	5713 ms
Graph-only (<code>graph_only</code>)	0.611	0.440	0.000	0	<1 ms
Hybrid (<code>hybrid</code>)	0.603	0.440	0.020	1	61 ms
Hybrid + LTL (<code>hybrid_ltl</code>)	0.611	0.440	0.000	1	61 ms
Full (<code>full</code>)	0.611	0.440	0.000	1	61 ms

Mean Latency is amortized wall time per window (total LLM time

divided by 100 windows). LLM-only invokes the 8B model on every window (≈ 5.7 s each); hybrid invokes it once (≈ 61 ms amortized). JSON: `darpa_ablation_gcp.json`.

TABLE VII

SENTINEL MITRE ATT&CK COVERAGE ACROSS THE 14 SIMULATION SCENARIOS. FALCO COVERAGE ($\checkmark$ /PARTIAL/ $\times$) REFLECTS THE DEFAULT COMMUNITY RULESET [?]; CUSTOM RULES COULD EXTEND FALCO'S COVERAGE FOR SOME TECHNIQUES. THIS IS A QUALITATIVE COMPARISON BASED ON THE CAPABILITIES DESCRIBED IN PUBLISHED DOCUMENTATION, NOT A HEAD-TO-HEAD BENCHMARK ON IDENTICAL WORKLOADS.

TTP ID	Technique	SENTINEL	Falco
T1059	Command Interpreter	$\checkmark$	$\checkmark$
T1003	Credential Dumping	$\checkmark$	$\checkmark$
T1071	C2 over App Protocol	$\checkmark$	partial
T1190	Public-facing Exploit	$\checkmark$	$\checkmark$
T1078	Valid Accounts	$\checkmark$	$\times$
T1210	Lateral Movement	$\checkmark$	partial
T1041	Exfiltration over C2	$\checkmark$	partial
T1055	Process Injection	$\checkmark$	$\times$
T1068	Privilege Escalation	$\checkmark$	$\checkmark$
T1562	Impair Defenses	$\checkmark$	$\times$
Coverage		10/10	6/10

TABLE VIII

RED-TEAM EVALUATION: 15 ATTACKER-AWARE EVASION SCENARIOS, ALL MALICIOUS GROUND TRUTH. “LAYER” = PRIMARY DETECTION MECHANISM: **H**=HARD-TRIGGER, **L**=LLM+ENTROPY, **T**=LTL GUARDIAN, **C**=PCABP, **LSM**=LSM POST-RESOLUTION. ✓ = DETECTED; — = WOULD EVADE THAT LAYER ALONE. EV-14*: DNS REBINDING; LLM LAYER (**L**) EXPECTED TO FLAG C2 IP RANGE.

ID	Strategy	H	L	T	C	LSM
EV-01	Entropy evasion (single sc-type)	✓	—	—	—	—
EV-02	Benign flood (1 attack in 14 benign)	✓	—	—	—	—
EV-03	Slow-and-low (1 signal/window)	✓	—	—	—	—
EV-04	Kill-chain split (2 PIDs)	✓	—	—	—	—
EV-05	LOLBins (python3→cat→shadow)	✓	—	—	—	—
EV-06	Obfuscated cmd (base64→shadow)	✓	—	—	—	—
EV-07	Time-bomb at window boundary	✓	—	—	—	—
EV-08	prctl masquerade (comm=nginx)	✓	—	✓	—	—
EV-09	Slow exfil (AX-2 temporal)	✓	—	✓	—	—
EV-10	PCABP nginx-mimicry (heap IP)	—	✓	—	✓	—
EV-11	TOCTOU race (symlink→shadow)	✓	—	—	—	✓
EV-12	Container escape (mount pivot)	—	—	✓	—	—
EV-13	Supply chain (pip post-install)	✓	—	✓	—	—
EV-14*	DNS rebinding (domain→C2 IP)	—	✓	—	—	—
EV-15	Multi-stage APT dormancy	✓	—	✓	—	—
Detection rate (GCP Ollama)						14/15 (93.3%)

EV-10–15 require PCABP, LSM, or LTL; EV-14 (DNS rebinding) evaded.

VIII. CONCLUSION

We presented SENTINEL, a kernel security framework that closes the semantic gap between low-level syscall telemetry and high-level attacker intent through five integrated contributions (Section ??). **LSM-eBPF co-design** replaces tracepoints with post-resolution LSM hooks, eliminating the TOCTOU evasion surface present in all prior eBPF detectors. **Graph-structured prompting via IPG** compresses multi-process syscall windows into structured YAML graphs—a novel paradigm that reduces LLM token consumption by 57.5% at $n=20$ events (tiktoken cl100k_base, 15 GCP trace files) while preserving causal semantics. **SSL/TLS uprobe-mediated AI agent monitoring** correlates LLM API intent with kernel actions inside the IPG, enabling detection of prompt-injection-driven APT chains invisible to syscall-only monitors. **Confidence-Weighted Adaptive Enforcement** maps continuous LLM confidence scores to five graduated kernel responses, replacing the binary block/allow model of all existing tools. Evaluated on 14 manually authored simulation scenarios using Llama-3.1-8B (8B-only path: 14/14 correct), and on 105 native-Linux strace traces (TPR=0.760, FPR=0.236 at production $\theta_{\text{low}}=1.2$, Acc=0.762; ungated LLM FPR=0.255), SENTINEL routes each detection to the appropriate CWAE enforcement tier — demonstrating that intent-aware, TOCTOU-resistant, AI-agent-aware zero-trust enforcement is deployable on commodity x86 Linux (CPU pipeline with optional GPU-accelerated local LLM escalation). The full implementation and datasets are publicly available at <https://github.com/jshaik-lab/ebpf-lsm-apt-monitor>.

DATA AVAILABILITY STATEMENT

Code, scenarios, strace corpora, PCABP weights, and evaluation scripts: <https://github.com/jshaik-lab/ebpf-lsm-apt-monitor> (Zenodo DOI upon acceptance). DARPA TC E3 CADETS: <https://github.com/darpa-i2o/Transparent-Computing>. GCP result JSONs and MANIFEST.json reproduce all cited metrics (§??).

ACKNOWLEDGMENTS

The authors thank the eBPF and Linux kernel communities for the BPF LSM framework and documentation. The author used Claude (Anthropic) for assistance with manuscript writing, editing, $\LaTeX$ formatting, and coding assistance. All research design, experimental methodology, system architecture, and reported results are solely the work of the author.

CONFLICTS OF INTEREST

The authors declare that there is no conflict of interest regarding the publication of this article.

REFERENCES

- [1] “DARPA Transparent Computing Program: Final Technical Report,” Defense Advanced Research Projects Agency (DARPA), Tech. Rep., 2020, program I2O. Available: <https://github.com/darpa-i2o/Transparent-Computing>.
- [2] OSSEC Project, “OSSEC: Open Source Host-Based Intrusion Detection System,” Open Source Project, 2023, version 3.7. Available: <https://www.ossec.net>.
- [3] S. Inc., “Falco: Cloud-Native Runtime Security,” *CNCF Open Source Project*, 2023, version 0.35. Available: <https://falco.org>.
- [4] Isovalent, “Tetragon: eBPF-Based Security Observability and Runtime Enforcement,” Open Source Project, 2023, version 1.1. Available: <https://tetragon.io>.
- [5] M. Du, F. Li, G. Zheng, and V. Srikumar, “DeepLog: Anomaly Detection and Diagnosis from System Logs Through Deep Learning,” in *Proc. ACM CCS*, 2017, pp. 1285–1298.
- [6] H. Guo, S. Yuan, and X. Wu, “LogBERT: Log Anomaly Detection via BERT,” in *Proc. IEEE IJCNN*, 2021, pp. 1–8.
- [7] Linux Kernel Documentation, “LSM BPF Programs: BPF_PROG_TYPE_LSM,” Linux Kernel Documentation, 2023, available: https://docs.kernel.org/bpf/prog_lsm.html.
- [8] Meta AI, “Meta Llama 3: The Most Capable Openly Available LLM to Date,” Meta AI Blog, 2024, available: <https://ai.meta.com/blog/meta-llama-3/>.
- [9] S. Hassan *et al.*, “WATSON: Abstracting Behaviors from Audit Logs via Aggregation of Contextually Similar Events,” in *Proc. NDSS*, 2020.
- [10] —, “WATSON: Abstracting Behaviors from Audit Logs via Aggregation of Contextually Similar Events (Extended),” in *Proc. NDSS*, 2023, extended version with improved GNN provenance reduction.
- [11] S. Han *et al.*, “UNICORN: Runtime Provenance-Based Detector for Advanced Persistent Threats,” in *Proc. NDSS*, 2020.
- [12] Q. Wang *et al.*, “You Are What You Do: Hunting Stealthy Malware via Data Provenance Analysis,” in *Proc. NDSS*, 2020.
- [13] X. Feng *et al.*, “DEPIMPACT: Identifying the Root Cause Node for Attack Investigation via Dependency Impact Analysis,” in *Proc. USENIX Security*, 2022, pp. 3203–3220.
- [14] Y. Chen *et al.*, “LLM-APTDS: A High-Precision Advanced Persistent Threat Detection System for Imbalanced Data Based on Large Language Models with Strong Interpretability,” *Future Generation Computer Systems*, 2025.
- [15] R. Patel *et al.*, “Enhancing Network Security Using Knowledge Graphs and Large Language Models for Explainable Threat Detection,” in *Future Generation Computer Systems*, 2025.
- [16] Y. Zheng, W. Xia, and H. Chen, “AgentSight: System-Level Observability for AI Agents Using eBPF,” arXiv:2508.02736, 2025, available: <https://arxiv.org/abs/2508.02736>.
- [17] X. Liu *et al.*, “Knowledge Transfer from LLMs to Provenance Analysis: A Semantic-Augmented Method for APT Detection,” arXiv:2503.18316, 2025, available: <https://arxiv.org/abs/2503.18316>.
- [18] G. Gerganov, “llama.cpp: Inference of LLaMA Model in Pure C/C++,” GitHub Repository, 2024, available: <https://github.com/ggerganov/llama.cpp>.